

CUSTOM RPC FRAMEWORK WITH CALCULATOR SERVICE

Understanding Remote Communication in Distributed Systems

PROJECT OVERVIEW

This project builds a custom communication system that enables two programs to interact over a network as if they were running locally on the same machine.

At its core, the framework features a Calculator Service that supports remote mathematical operations – including addition, subtraction, multiplication, and division – executed seamlessly across a network connection.

Unlike solutions that rely on third-party libraries, this entire RPC framework is built from scratch in Python, providing a deep, hands-on understanding of how remote communication works at every layer.

The project demonstrates the client-server model in action: a client sends a request, the server processes it remotely, and the result is returned – all transparently, as if the operation were local.

THE PROBLEM BEING SOLVED

"Modern applications often run distributed across multiple machines"

- 1.Services and data may be remote – not always accessible locally
- 2.Continuous communication is needed between distributed systems
- 3.Local execution is not possible for remote services

Solution: A client-server request-response model over a network bridges the gap, allowing programs to interact seamlessly regardless of location.



PURPOSE OF THE PROJECT

Understand remote
communication
fundamentals

Build a custom RPC
framework from scratch

Simulate distributed
system operations

Learn how requests and
responses travel through
a network

Explore reliability
mechanisms in
communication

Demonstrate remote
service access with a
simple calculator
example

REAL-WORLD EXAMPLE: BANKING MOBILE APP

A banking mobile app is a perfect analogy for RPC in action. When you check your account balance, your phone sends a request over the network to the bank's remote server. The server processes the request and returns the result – your balance – back to your phone. This mirrors exactly how an RPC framework operates: a client calls a remote function and receives the result as if it were local.

How It Works:

- 📱 Phone sends a request → identifies the operation and parameters
 - 🌐 Request travels over the network to the bank server
 - 💻 Server processes the request and executes the function
 - ✉️ Result returns to the phone seamlessly
- "Requesting account balance remotely mirrors how RPC frameworks work."



MAIN FEATURES

Remote Communication

Enables programs to interact over a TCP network as if running locally – bridging the gap between distributed systems seamlessly.

Calculator Service

Supports core mathematical operations remotely: add, subtract, multiply, and divide – executed on the server, results returned to the client.

Automatic Request & Response

Handles the full lifecycle of remote calls automatically – packaging requests, transmitting them, and returning results without manual intervention.

Seamless Interaction

Clients invoke remote functions as if they were local, abstracting all network complexity behind a clean and simple interface.

TECHNOLOGIES USED

Python Programming Language

The core implementation language for the entire RPC framework. Python's simplicity and rich standard library make it ideal for building custom network communication systems from scratch, without relying on external RPC libraries.

TCP Sockets for Network Communication

TCP (Transmission Control Protocol) sockets form the backbone of the framework's network layer. They provide reliable, ordered, and error-checked delivery of data between the client and server across the network.

EXPECTED OUTCOMES

Hands-on experience with distributed systems principles, gaining practical knowledge of how programs communicate across networks and understanding the mechanics behind client-server interactions in real-world environments.

Understanding client-server architecture and protocol design, including how requests are structured, transmitted, processed, and how responses are returned reliably across a TCP network connection.

Insight into building reliability features in network communication and knowledge of software architecture for remote services, laying a strong foundation for future work in distributed and cloud-based systems.

EDUCATIONAL ∇ VALUE

Foundational concepts powering today's most critical technologies:

- Cloud Computing & Microservices – RPC frameworks underpin how cloud platforms coordinate distributed services at scale
- Banking & Mobile Apps – Remote communication principles drive real-time transactions and account management across networks
- Distributed Platform Design – Exposure to how large-scale systems like e-commerce and streaming platforms are architected
- Practical Skills for Enterprise Systems – Applicable knowledge for building online services, APIs, and service-oriented architectures
- Encourages Experimentation – Deep understanding of remote communication opens pathways to advanced distributed system design

LOCAL VS REMOTE EXECUTION

Local Execution:

- Operations performed on the same machine
- Direct function calls with no network involved
- Immediate results with minimal latency
- Simple and straightforward execution flow

Remote Execution via RPC:

- Requests sent over network to a remote system
- Server processes and sends back results
- Adds complexity but enables distributed architectures
- Powers modern cloud and enterprise systems



SUMMARY & CLOSING

The RPC framework demonstrates how remote communication works at a fundamental level, enabling two programs to interact over a network as if running locally. The calculator service provides a simple, clear example of remote procedure calls in action.

This project builds foundational skills for understanding modern distributed software systems – from client-server architecture and protocol design to reliability mechanisms and data serialization. It encourages further exploration of networking, protocols, and distributed design.

REMOTE PROCEDURE CALL (RPC)
FRAMEWORK

THANK YOU!

QUESTIONS & DISCUSSION